

C Programming Exercises

Advanced Level — Line-by-Line Explanations

Name: PATHIPATI
RUTHWIK

Roll No: CH.SC.U4CSE25230

Date: 29/06/2026

Instructions: Read each program carefully, study the line-by-line explanation, trace through the code by hand, and write the expected output. Then check your answer with the Expected Output provided.

Exercise 1: Formatted Output with Multiple Variables

Problem:

Write a C program that declares an integer, a float, and a character variable. Print all three on a single line using formatted output (printf with format specifiers). Also print their memory sizes using sizeof().

Code:

```
#include <stdio.h>
int main() {
    int age = 21;
    float gpa = 8.75f;
    char grade = 'A';
    printf("Age = %d, GPA = %.2f, Grade = %c\n", age, gpa, grade);
    printf("Size of int: %lu bytes\n", sizeof(int));
    printf("Size of float: %lu bytes\n", sizeof(float));
    printf("Size of char: %lu bytes\n", sizeof(char));
    return 0;
}
```

Line-by-Line Explanation:

1. `#include <stdio.h>` includes the standard input/output library, which provides `printf()` and `scanf()` functions needed for console I/O operations.
2. `int main()` is the entry point of every C program. The operating system calls this function first when the program is executed. The `int` return type means the function returns an integer status code.
3. `int age = 21;` declares an integer variable named "age" and initializes it with the value 21. The `int` data type typically uses 4 bytes of memory and stores whole numbers.
4. `float gpa = 8.75f;` declares a floating-point variable named "gpa" and initializes it with 8.75. The "f" suffix tells the compiler to treat 8.75 as a float (single precision) rather than a double (double precision).

5. `char grade = 'A';` declares a character variable named "grade" and stores the character 'A'. Single quotes are used to represent character literals in C. A char occupies 1 byte of memory.
6. `printf("Age = %d, GPA = %.2f, Grade = %c\n", age, gpa, grade);` prints the values of all three variables on one line. `%d` is the format specifier for int, `%.2f` prints a float with exactly 2 decimal places, and `%c` prints a single character. `\n` moves the cursor to the next line.
7. `printf("Size of int: %lu bytes\n", sizeof(int));` uses the `sizeof()` operator to find how many bytes an int occupies in memory. `%lu` is the format specifier for "unsigned long", which is the return type of `sizeof()`.
8. `printf("Size of float: %lu bytes\n", sizeof(float));` prints the memory size of a float variable, which is typically 4 bytes on most modern systems.
9. `printf("Size of char: %lu bytes\n", sizeof(char));` prints the memory size of a char variable, which is always 1 byte on all systems, guaranteed by the C standard.
10. `return 0;` signals to the operating system that the program terminated successfully. A return value of 0 conventionally means no errors occurred.
11. The closing brace `}` marks the end of the `main()` function body and the end of the program.

Your Output:

Output

```
Age = 21, GPA = 8.75, Grade = A
Size of int: 4 bytes
Size of float: 4 bytes
Size of char: 1 bytes
```

Expected Output:

```
Age = 21, GPA = 8.75, Grade = A
Size of int: 4 bytes
Size of float: 4 bytes
Size of char: 1 bytes
```

Exercise 2: Swap Two Numbers and Calculate Sum & Average

Problem:

Write a C program that reads two integers from the user, swaps their values without using a third variable, and then prints the swapped values along with their sum and average.

Code:

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    printf("Before swap: a = %d, b = %d\n", a, b);
    a = a + b;
    b = a - b;
    a = a - b;
    printf("After swap: a = %d, b = %d\n", a, b);
    int sum = a + b;
    float avg = (float)sum / 2;
    printf("Sum = %d, Average = %.2f\n", sum, avg);
    return 0;
}
```

Line-by-Line Explanation:

1. `#include <stdio.h>` includes the standard I/O library for `printf()` and `scanf()` functions.
2. `int main()` is the program entry point. Execution begins here.
3. `int a, b;` declares two integer variables "a" and "b" without initializing them. They will hold whatever garbage value is in memory until `scanf()` assigns values to them.
4. `printf("Enter two numbers: ");` prompts the user to type two numbers. No newline is printed, so the cursor stays on the same line waiting for input.
5. `scanf("%d %d", &a, &b);` reads two integer values from the keyboard. `%d` is the format specifier for integers. The `&` (address-of) operator gives `scanf()` the memory address of each variable so it can store the input values directly.
6. `printf("Before swap: a = %d, b = %d\n", a, b);` displays the original values of a and b before the swap operation, so the user can verify the swap worked correctly.
7. `a = a + b;` adds the value of b to a. Now "a" holds the combined sum of both original values. This is the first step of the arithmetic swap technique.
8. `b = a - b;` subtracts the new b (which is still the original b) from the new a (which is the sum). This gives the original value of a, effectively assigning it to b.
9. `a = a - b;` subtracts the new b (which is now the original a) from the current a (which is the sum). This gives the original value of b, completing the swap without any temporary variable.
10. `printf("After swap: a = %d, b = %d\n", a, b);` displays the values after swapping to confirm the swap was successful.
11. `int sum = a + b;` calculates the sum of the two (now swapped) numbers and stores it in a new integer variable "sum".
12. `float avg = (float)sum / 2;` divides the sum by 2 to get the average. The `(float)` cast is essential — without it, integer division would truncate the result. For example, if sum is 15, $(float)15 / 2 = 7.50$, but $15 / 2 = 7$ (truncated).

13. `printf("Sum = %d, Average = %.2f\n", sum, avg);` prints both the sum (as integer) and the average (as float with 2 decimal places).
14. `return 0;` indicates the program ended successfully.
15. The closing brace `}` ends the `main()` function.

Your Output:

Output

```
Enter two numbers: 12 28
Before swap: a = 12, b = 28
After swap: a = 28, b = 12
Sum = 40, Average = 20.00
```

Expected Output:

```
Enter two numbers: 10 25
Before swap: a = 10, b = 25
After swap: a = 25, b = 10
Sum = 35, Average = 17.50
```

Exercise 3: Number Classification Using Bitwise and Modulo Operations

Problem:

Write a C program that reads an integer and classifies it using bitwise AND for even/odd check, and also checks divisibility by 3 and by 5. Print all classification results.

Code:

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n & 1)
        printf("%d is Odd\n", n);
    else
        printf("%d is Even\n", n);
    if (n % 3 == 0)
        printf("%d is divisible by 3\n", n);
    else
        printf("%d is NOT divisible by 3\n", n);
    if (n % 5 == 0)
        printf("%d is divisible by 5\n", n);
    else
        printf("%d is NOT divisible by 5\n", n);
    return 0;
}
```

Line-by-Line Explanation:

1. `#include <stdio.h>` includes the standard I/O library for input/output functions.
2. `int main()` is the entry point of the C program.
3. `int n;` declares an integer variable "n" to store the user input.
4. `printf("Enter a number: ");` displays a prompt asking the user to enter a number.
5. `scanf("%d", &n);` reads an integer value from the keyboard and stores it at the memory address of variable n.
6. `if (n & 1)` uses a bitwise AND operation to check the least significant bit of n. In binary, all odd numbers end with 1 and all even numbers end with 0. If `n & 1` equals 1 (true), the number is odd. This method is faster than using the modulo operator.
7. `printf("%d is Odd\n", n);` executes when the bitwise condition is true, confirming the number is odd.
8. `else` is the alternative branch that runs when `n & 1` equals 0 (false), meaning the number is even.
9. `printf("%d is Even\n", n);` prints that the number is even.
10. `if (n % 3 == 0)` uses the modulo operator to check if n is perfectly divisible by 3. If `n % 3` yields a remainder of 0, then 3 is a factor of n.
11. `printf("%d is divisible by 3\n", n);` confirms that 3 divides the number evenly.
12. `else` handles the case where `n % 3` is not zero, meaning 3 is not a factor.
13. `printf("%d is NOT divisible by 3\n", n);` reports that the number is not divisible by 3.
14. `if (n % 5 == 0)` similarly checks divisibility by 5 using the modulo operator.

15. `printf("%d is divisible by 5\n", n);` prints when 5 divides the number evenly.
16. `else` handles the case where 5 is not a factor of the number.
17. `printf("%d is NOT divisible by 5\n", n);` reports that the number is not divisible by 5.
18. `return 0;` signals successful program termination.
19. The closing brace `}` marks the end of the program.

Your Output:

Output

```
Enter a number: 1228
1228 is Even
1228 is NOT divisible by 3
1228 is NOT divisible by 5
```

Expected Output:

```
Enter a number: 15
15 is Odd
15 is divisible by 3
15 is divisible by 5
```

Exercise 4: Factorial using Recursion with Input Validation

Problem:

Write a C program that calculates the factorial of a number using a recursive function. Include input validation to reject negative numbers and handle the edge case of $0! = 1$.

Code:

```
#include <stdio.h>
long factorial(int n) {
    if (n < 0) return -1;
    if (n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
}
int main() {
    int num;
    printf("Enter a non-negative integer: ");
    scanf("%d", &num);
    if (num < 0) {
        printf("Error: Factorial is not defined for negative numbers.\n");
    } else {
        printf("Factorial of %d = %ld\n", num, factorial(num));
    }
    return 0;
}
```

Line-by-Line Explanation:

1. `#include <stdio.h>` includes the standard I/O library for input and output operations.
2. `long factorial(int n)` defines a recursive function named "factorial" that takes an integer parameter "n" and returns a long integer. The long type is used because factorials grow very quickly and can exceed the range of int.
3. `if (n < 0) return -1;` is a safety check inside the recursive function. If a negative number somehow reaches this function, it returns -1 as an error code to prevent infinite recursion.
4. `if (n == 0 || n == 1) return 1;` is the base case of the recursion. The factorial of both 0 and 1 is defined as 1. Without a base case, the recursion would never stop, causing a stack overflow.
5. `return n * factorial(n - 1);` is the recursive case. The function calls itself with n-1, multiplying the current value of n by the factorial of n-1. For example, $\text{factorial}(5) = 5 * \text{factorial}(4) = 5 * 4 * \text{factorial}(3)$, and so on until the base case is reached.
6. `int main()` is the program entry point where execution begins.
7. `int num;` declares an integer variable "num" to store the user input.
8. `printf("Enter a non-negative integer: ");` prompts the user to enter a non-negative integer.
9. `scanf("%d", &num);` reads the integer input from the user and stores it in the variable num.
10. `if (num < 0) {` checks if the user entered a negative number, which is invalid for factorial calculation since factorials are defined only for non-negative integers.
11. `printf("Error: Factorial is not defined for negative numbers.\n");` prints an error message if the input is negative, preventing the program from attempting an invalid calculation.
12. `} else {` begins the else block, which runs when the input is valid (zero or positive).
13. `printf("Factorial of %d = %ld\n", num, factorial(num));` calls the factorial function with the user input and prints the result. `%ld` is the format specifier for long integers.

14. } closes the else block.
15. return 0; indicates the program finished successfully.
16. The closing brace } ends the main() function.

Your Output:

Output

```
Enter a non-negative integer: 5
Factorial of 5 = 120
```

Expected Output:

```
Enter a non-negative integer: 6
Factorial of 6 = 720
```

Exercise 5: Sum of Even and Odd Numbers Separately up to N

Problem:

Write a C program that reads a positive integer N, then calculates and prints the sum of all even numbers from 1 to N and the sum of all odd numbers from 1 to N using a while loop.

Code:

```
#include <stdio.h>
int main() {
    int n, i = 1;
    int evenSum = 0, oddSum = 0;
    printf("Enter N: ");
    scanf("%d", &n);
    while (i <= n) {
        if (i % 2 == 0)
            evenSum += i;
        else
            oddSum += i;
        i++;
    }
    printf("Sum of even numbers (1 to %d) = %d\n", n, evenSum);
    printf("Sum of odd numbers (1 to %d) = %d\n", n, oddSum);
    printf("Total sum = %d\n", evenSum + oddSum);
    return 0;
}
```


Line-by-Line Explanation:

1. `#include <stdio.h>` includes the standard I/O library for `printf()` and `scanf()`.
2. `int main()` is the program entry point.
3. `int n, i = 1;` declares two integer variables: "n" for the upper limit (uninitialized) and "i" for the loop counter, initialized to 1.
4. `int evenSum = 0, oddSum = 0;` declares and initializes two accumulator variables to zero. `evenSum` will collect the sum of all even numbers and `oddSum` will collect the sum of all odd numbers.
5. `printf("Enter N: ");` prompts the user to enter the upper limit N.
6. `scanf("%d", &n);` reads the integer value from the user and stores it in n.
7. `while (i <= n) {` starts a while loop that continues as long as the counter i is less than or equal to n. The loop will iterate from 1 through N inclusive.
8. `if (i % 2 == 0)` checks if the current value of i is even by testing if the remainder when divided by 2 equals zero.
9. `evenSum += i;` adds the current number i to the `evenSum` accumulator if i is even. This is shorthand for `evenSum = evenSum + i`.
10. `else` handles the case where i is odd (`i % 2 != 0`).
11. `oddSum += i;` adds the current number i to the `oddSum` accumulator if i is odd.
12. `i++;` increments the loop counter by 1 after each iteration. Without this, the loop would run infinitely since i would never change.
13. `}` closes the while loop body.
14. `printf("Sum of even numbers (1 to %d) = %d\n", n, evenSum);` prints the accumulated sum of all even numbers from 1 to N.
15. `printf("Sum of odd numbers (1 to %d) = %d\n", n, oddSum);` prints the accumulated sum of all odd numbers from 1 to N.
16. `printf("Total sum = %d\n", evenSum + oddSum);` prints the combined total of both sums, which should equal $N*(N+1)/2$ (the formula for the sum of the first N natural numbers).
17. `return 0;` signals successful program completion.
18. The closing brace `}` ends the main function.

Your Output:

Output

```
Enter N: 5
Sum of even numbers (1 to 5) = 6
Sum of odd numbers (1 to 5) = 9
Total sum = 15
```

Expected Output:

```
Enter N: 10
Sum of even numbers (1 to 10) = 30
Sum of odd numbers (1 to 10) = 25
Total sum = 55
```

Exercise 6: Multiplication Table Grid using Nested Loops

Problem:

Write a C program that reads a number N and prints a multiplication table grid from 1 to N using nested for loops. Each row should show products of the row number with columns 1 through N.

Code:

```
#include <stdio.h>
int main() {
    int n, i, j;
    printf("Enter N: ");
    scanf("%d", &n);
    printf(" ");
    for (j = 1; j <= n; j++)
        printf("%4d", j);
    printf("\n");
    for (i = 1; i <= n; i++) {
        printf("%4d", i);
        for (j = 1; j <= n; j++)
            printf("%4d", i * j);
        printf("\n");
    }
    return 0;
}
```

Line-by-Line Explanation:

1. `#include <stdio.h>` includes the standard I/O library.
2. `int main()` is the entry point of the program.
3. `int n, i, j;` declares three integer variables: `n` for the table size, `i` for the outer (row) loop, and `j` for the inner (column) loop.
4. `printf("Enter N: ");` prompts the user for the table size.
5. `scanf("%d", &n);` reads the integer `N` from the user.
6. `printf(" ");` prints 4 spaces to leave room for the row header column on the left side of the grid.
7. `for (j = 1; j <= n; j++)` starts a for loop that prints the column header numbers (1 through `N`) across the top of the table.

8. `printf("%4d", j);` prints each column number `j` using a field width of 4 characters (`%4d`), which ensures all numbers are right-aligned in columns of equal width for a neat grid.
9. `printf("\n");` prints a newline after the column headers to move to the next row.
10. `for (i = 1; i <= n; i++) {` begins the outer for loop. Each iteration of this loop produces one row of the multiplication table, with `i` representing the current row number.
11. `printf("%4d", i);` prints the row header (the current value of `i`) at the start of each row, aligned to a width of 4 characters.
12. `for (j = 1; j <= n; j++)` begins the inner for loop. This loop iterates through all columns (1 through `N`) for the current row.
13. `printf("%4d", i * j);` calculates and prints the product of `i` and `j`. The `%4d` format ensures each product occupies exactly 4 character positions, creating a clean grid alignment.
14. `printf("\n");` prints a newline after completing each row, moving the cursor to the beginning of the next line.
15. `}` closes the outer for loop body.
16. `return 0;` indicates successful program termination.
17. The closing brace `}` ends the main function.

Your Output:

Output				
Enter N: 4				
	1	2	3	4
1	1	2	3	4
2	2	4	6	8
3	3	6	9	12
4	4	8	12	16

Expected Output:

```

Enter N: 5
  1  2  3  4  5
1  1  2  3  4  5
2  2  4  6  8 10
3  3  6  9 12 15
4  4  8 12 16 20
5  5 10 15 20 25

```

Exercise 7: Check if a Number is a Palindrome

Problem:

Write a C program that reads an integer and checks whether it is a palindrome (reads the same forwards and backwards). Use a while loop to reverse the number and compare it with the original.

Code:

```
#include <stdio.h>
int main() {
    int n, original, rev = 0, digit;
    printf("Enter a number: ");
    scanf("%d", &n);
    original = n;
    while (n > 0) {
        digit = n % 10;
        rev = rev * 10 + digit;
        n = n / 10;
    }
    if (original == rev)
        printf("%d is a Palindrome\n", original);
    else
        printf("%d is NOT a Palindrome\n", original);
    return 0;
}
```

Line-by-Line Explanation:

1. `#include <stdio.h>` includes the standard I/O library for `printf()` and `scanf()`.
2. `int main()` is the program entry point.
3. `int n, original, rev = 0, digit;` declares four integer variables: "n" for user input, "original" to store a copy of the input, "rev" initialized to 0 to build the reversed number, and "digit" to store each extracted digit.
4. `printf("Enter a number: ");` prompts the user to enter a number.
5. `scanf("%d", &n);` reads the integer value from the user and stores it in n.
6. `original = n;` saves a copy of the original number in the variable "original". This is necessary because n will be modified (divided down to 0) inside the while loop, and we need the original value for the final comparison.
7. `while (n > 0) {` starts a loop that continues as long as n is greater than 0. The loop processes each digit of n from right to left, and stops when all digits have been extracted.
8. `digit = n % 10;` extracts the last digit of n using the modulo operator. For example, if n is 121, then `121 % 10` gives 1 (the rightmost digit).
9. `rev = rev * 10 + digit;` appends the extracted digit to the right side of the reversed number. Multiplying rev by 10 shifts its digits left by one place, then adding digit places the new digit in the ones place. For example, if rev is 12 and digit is 1, rev becomes 121.

10. `n = n / 10`; removes the last digit from `n` using integer division. For example, `121 / 10` gives 12 (the decimal part is truncated). This progressively strips digits from right to left.
11. `}` closes the while loop body. When the loop finishes, `rev` holds the complete reversed number and `n` is 0.
12. `if (original == rev)` compares the original number with the reversed number. If they are identical, the number reads the same forwards and backwards, meaning it is a palindrome.
13. `printf("%d is a Palindrome\n", original)`; prints that the number is a palindrome if the comparison is true.
14. `else` handles the case where the original and reversed numbers are different.
15. `printf("%d is NOT a Palindrome\n", original)`; prints that the number is not a palindrome.
16. `return 0`; signals successful program termination.
17. The closing brace `}` ends the main function.

Your Output:

```
Output
Enter a number: 1231
1231 is NOT a Palindrome
```

Expected Output:

```
Enter a number: 12321
12321 is a Palindrome
```

Exercise 8: Print All Prime Numbers Up to N

Problem:

Write a C program that reads a positive integer `N` and prints all prime numbers from 2 to `N`. Use an optimized check that tests divisors only up to the square root of each number.

Code:

```
#include <stdio.h>
#include <math.h>
int isPrime(int num) {
    if (num <= 1) return 0;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) return 0;
    }
    return 1;
}
int main() {
    int n;
    printf("Enter N: ");
    scanf("%d", &n);
    printf("Prime numbers up to %d:\n", n);
    for (int i = 2; i <= n; i++) {
        if (isPrime(i))
            printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

Line-by-Line Explanation:

1. `#include <stdio.h>` includes the standard I/O library for input and output functions.
2. `#include <math.h>` includes the math library, which provides mathematical functions like `sqrt()`. It is included here because the prime-checking logic uses `i * i <= num`, which is equivalent to checking up to the square root.
3. `int isPrime(int num)` defines a helper function that checks if a given number "num" is prime. It returns 1 (true) if the number is prime, and 0 (false) if it is not.
4. `if (num <= 1) return 0;` immediately returns 0 (not prime) for numbers less than or equal to 1, since 0 and 1 are not prime numbers by mathematical definition.
5. `for (int i = 2; i * i <= num; i++) {` starts a for loop that checks potential divisors starting from 2. The condition `i * i <= num` is an optimization: if a number has a factor larger than its square root, it must also have a factor smaller than its square root. So we only need to check up to the square root.
6. `if (num % i == 0) return 0;` checks if num is divisible by i. If it is, num has a factor other than 1 and itself, so it is not prime. The function immediately returns 0.
7. `}` closes the for loop. If the loop completes without finding any divisors, the number is prime.
8. `return 1;` returns 1 (true) indicating the number is prime, since no divisors were found.
9. `int main()` is the program entry point.
10. `int n;` declares an integer variable n for the upper limit.

11. `printf("Enter N: ");` prompts the user to enter the upper limit.
12. `scanf("%d", &n);` reads the integer from the user.
13. `printf("Prime numbers up to %d:\n", n);` prints a header line indicating which primes will be listed.
14. `for (int i = 2; i <= n; i++) {` starts a loop from 2 to N. Each number in this range will be tested for primality. We start at 2 because 0 and 1 are not prime.
15. `if (isPrime(i))` calls the `isPrime()` function for the current value of `i`. If it returns 1 (true), the number is prime.
16. `printf("%d ", i);` prints the prime number followed by a space.
17. `}` closes the for loop.
18. `printf("\n");` prints a final newline after all primes have been listed.
19. `return 0;` indicates the program finished successfully.
20. The closing brace `}` ends the main function.

Your Output:

Output

```
Enter N: 5
Prime numbers up to 5:
2 3 5
```

Expected Output:

```
Enter N: 30
Prime numbers up to 30:
2 3 5 7 11 13 17 19 23 29
```

Exercise 9: Fibonacci Series Between a Given Range

Problem:

Write a C program that reads two integers (lower and upper bounds) and prints all Fibonacci numbers that fall within that range, inclusive. Do not use recursion; use an iterative approach.

Code:

```
#include <stdio.h>
int main() {
    int low, high;
    printf("Enter lower and upper bound: ");
    scanf("%d %d", &low, &high);
    int a = 0, b = 1, next;
    printf("Fibonacci numbers between %d and %d:\n", low, high);
    while (a <= high) {
        if (a >= low)
            printf("%d ", a);
        next = a + b;
        a = b;
        b = next;
    }
    printf("\n");
    return 0;
}
```

Line-by-Line Explanation:

1. `#include <stdio.h>` includes the standard I/O library.
2. `int main()` is the program entry point.
3. `int low, high;` declares two integer variables to store the lower and upper bounds of the range.
4. `printf("Enter lower and upper bound: ");` prompts the user to enter both bounds.
5. `scanf("%d %d", &low, &high);` reads two integers from the user. The first value goes into "low" and the second into "high".
6. `int a = 0, b = 1, next;` initializes the first two Fibonacci numbers: $a = 0$ (F0) and $b = 1$ (F1). The variable "next" will hold the next Fibonacci number during iteration.
7. `printf("Fibonacci numbers between %d and %d:\n", low, high);` prints a header showing the selected range.
8. `while (a <= high) {` starts a loop that continues generating Fibonacci numbers as long as the current value "a" is within or below the upper bound. Once "a" exceeds "high", all relevant Fibonacci numbers have been found.
9. `if (a >= low)` checks if the current Fibonacci number "a" is greater than or equal to the lower bound. This ensures only numbers within the specified range are printed.
10. `printf("%d ", a);` prints the current Fibonacci number followed by a space, but only if it falls within the user-specified range.
11. `next = a + b;` calculates the next Fibonacci number by adding the two previous numbers (a and b) together. This is the core formula of the Fibonacci sequence: $F(n) = F(n-1) + F(n-2)$.
12. `a = b;` shifts the values forward: the second number (b) becomes the first number (a) for the next iteration.

13. `b = next`; completes the shift: the newly calculated "next" value becomes the second number (b) for the next iteration.
14. `}` closes the while loop.
15. `printf("\n");` prints a newline after all Fibonacci numbers in the range have been displayed.
16. `return 0;` signals successful program termination.
17. The closing brace `}` ends the main function.

Your Output:

Output
Enter lower and upper bound: 1 10 Fibonacci numbers between 1 and 10: 1 1 2 3 5 8

Expected Output:

```
Enter lower and upper bound: 10 100
Fibonacci numbers between 10 and 100:
13 21 34 55 89
```

Exercise 10: GCD, LCM, and Coprime Check using Euclidean Algorithm

Problem:

Write a C program that reads two positive integers, finds their GCD using the Euclidean algorithm (iterative), calculates their LCM using the formula $LCM = (a * b) / GCD$, and checks whether the two numbers are coprime ($GCD = 1$).

Code:

```
#include <stdio.h>
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
int main() {
    int x, y;
    printf("Enter two numbers: ");
    scanf("%d %d", &x, &y);
    int g = gcd(x, y);
    int lcm = (x * y) / g;
    printf("GCD of %d and %d = %d\n", x, y, g);
    printf("LCM of %d and %d = %d\n", x, y, lcm);
    if (g == 1)
        printf("%d and %d are Coprime\n", x, y);
    else
        printf("%d and %d are NOT Coprime (GCD = %d)\n", x, y, g);
    return 0;
}
```

Line-by-Line Explanation:

1. `#include <stdio.h>` includes the standard I/O library for input and output functions.
2. `int gcd(int a, int b)` defines a function that computes the Greatest Common Divisor of two integers using the iterative Euclidean algorithm. It takes two integer parameters and returns an integer.
3. `while (b != 0) {` starts a loop that repeatedly replaces the pair (a, b) with $(b, a \% b)$ until b becomes 0. When b reaches 0, a holds the GCD.
4. `int temp = b;` saves the current value of b in a temporary variable. This is necessary because b is about to be overwritten in the next line, but we still need its original value to assign to a .
5. `b = a % b;` calculates the remainder of a divided by b , and stores it in b . This is the core step of the Euclidean algorithm: the GCD of (a, b) is the same as the GCD of $(b, a \% b)$.
6. `a = temp;` assigns the old value of b (saved in `temp`) to a . Now the pair (a, b) has been updated to $(\text{old_}b, a \% \text{old_}b)$ for the next iteration.
7. `}` closes the while loop. When the loop exits, b is 0 and a contains the GCD.
8. `return a;` returns the computed GCD to the calling function.
9. `int main()` is the program entry point.
10. `int x, y;` declares two integer variables to store the user input numbers.

11. `printf("Enter two numbers: ");` prompts the user to enter two numbers.
12. `scanf("%d %d", &x, &y);` reads two integers from the user and stores them in `x` and `y`.
13. `int g = gcd(x, y);` calls the `gcd()` function with the two user-provided numbers and stores the result in variable `g`.
14. `int lcm = (x * y) / g;` calculates the Least Common Multiple using the mathematical relationship: $\text{LCM}(a, b) = (a * b) / \text{GCD}(a, b)$. Since we already have the GCD, this is a simple division. Note: `x * y` is computed first, then divided by `g`.
15. `printf("GCD of %d and %d = %d\n", x, y, g);` prints the computed GCD of the two numbers.
16. `printf("LCM of %d and %d = %d\n", x, y, lcm);` prints the computed LCM of the two numbers.
17. `if (g == 1)` checks if the GCD is 1. Two numbers are called "coprime" (or "relatively prime") if their only common divisor is 1.
18. `printf("%d and %d are Coprime\n", x, y);` prints that the numbers are coprime if GCD equals 1.
19. `else` handles the case where GCD is greater than 1.
20. `printf("%d and %d are NOT Coprime (GCD = %d)\n", x, y, g);` prints that the numbers are not coprime and shows the actual GCD value.
21. `return 0;` signals successful program termination.
22. The closing brace `}` ends the main function.

Your Output:

Output

```
Enter two numbers: 12 28
GCD of 12 and 28 = 4
LCM of 12 and 28 = 84
12 and 28 are NOT Coprime (GCD = 4)
```

Expected Output:

```
Enter two numbers: 35 18
GCD of 35 and 18 = 1
LCM of 35 and 18 = 630
35 and 18 are Coprime
```